Artificial Intelligence

BS (CS) Spring 2026

Lab 05 Manual



Learning Objectives:

1. Informed searches
2. A* Search
3. Greedy Best-First Search

Lab Manual

Informed Search:

While uninformed search algorithms explore the search space without additional knowledge, informed search algorithms use **heuristic information** to guide the search process more effectively.
Informed search algorithms are essential in solving complex AI problems, such as pathfinding in robotics or game development, where optimal solutions must be found efficiently. By leveraging heuristics, informed algorithms can reduce the search space, improving speed and performance.

Heauristic Function

A heuristic is a function that estimates the cost of reaching the goal from a given node, providing additional information that helps the algorithm make smarter decisions.

Role of Heuristics

- Heuristics play a crucial role in informed search algorithms. They help prioritize which nodes or paths the algorithm should explore first by estimating how close a node is to the goal. This dramatically reduces the number of states explored, making the search process more efficient.
- Heuristic functions, typically denoted as **h(n)**, estimate the cost from a node to the goal. A well-chosen heuristic can greatly enhance the efficiency of the search by guiding the algorithm toward the goal more directly.
- The value of the **heuristic function is always positive.**
- Admissibility of the heuristic function is given as:

  $$h(n) \leq h*(n)$$

  Here $\boldsymbol{h(n)}$ is heuristic cost, and $\boldsymbol{h*(n)}$ is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.
- **Example of Informed Search**: The A* algorithm is an informed search algorithm that uses both the cost to reach a node and an estimated cost to the goal to prioritize nodes.

Informed search algorithms offer the advantage of being more targeted, allowing them to find solutions faster by focusing only on the most promising paths.

Types:

1. Greedy Best-First Search
2. A* Algorithm
3. Iterative Deepening A*
4. Hill Climbing

A* Algorithm

A* is a widely used informed search algorithm that efficiently finds the shortest path between two nodes in a graph. It uses a heuristic function to estimate the cost of reaching the goal from a given node.
It improves on earlier algorithms like **Dijkstra's Algorithm** by incorporating a **heuristic** that estimates the cost of reaching the goal from any given node, making it more efficient in terms of computational time.

Heuristic Function

$$f(n) = g(n) + h(n)$$

f(n): Estimated cost of the cheapest solution
g(n): Cost to reach node n from start state.
h(n): Cost to reach from node n to goal node.

Step-by-Step Guide

1. **Initialize the Open and Closed Lists**:
   - Open List contains nodes to be evaluated.
   - Closed List contains nodes that have already been evaluated.
2. **Add the start node to the Open List**.
3. **Loop until the Open List is empty or the goal is reached**:
   - Select the node from the Open List with the lowest **f(n)**, where **f(n) = g(n) + h(n)**.
   - If the current node is the goal, reconstruct the path and return it.
4. **For the selected node**:
   - Remove it from the Open List and add it to the Closed List.
   - Evaluate all **neighboring nodes**:
     - If a neighbor is in the Closed List, skip it.
     - If a neighbor is in the Open List, check if this path is better (lower **g(n)**); update if necessary.
     - Otherwise, calculate **g(n)**, **h(n)**, and **f(n)**, then add the neighbor to the Open List.
5. **End the loop** once the goal is reached or the Open List is empty (no path found).
6. **Return the reconstructed path** or indicate that no valid path exists.

Common Heuristics in A*

Several heuristics can be used in A*, depending on the structure of the problem:

1. Manhattan Distance:

Used in grid-based problems, where movements are restricted to horizontal and vertical directions.
Formula:

$$h(n) = |x_{goal} - x_{current}| + |y_{goal} - y_{current}|$$

2. Euclidean Distance:

Used when movement is allowed in any direction, accounting for the straight-line distance between two points.
Formula:

$$h(n) = \sqrt{(x_{goal} - x_{current})^2 + (y_{goal} - y_{current})^2}$$

3. Diagonal Distance:

Suitable for grid-based systems that allow diagonal movement.
Formula:

$$h(n) = \max(|x_{goal} - x_{current}|, |y_{goal} - y_{current}|)$$

Choosing the Right Heuristic

- **Manhattan Distance** is ideal for problems with grid-based layouts and no diagonal movements.
- **Euclidean Distance** is suitable when diagonal or any-direction movement is allowed.
- **Diagonal Distance** is used for grid problems with diagonal movements allowed.

Choosing the right heuristic depends on the nature of the environment and the problem at hand.

Implementation

```python
from heapq import heappop, heappush

def a_star_search(graph: dict, start: str, goal: str, heuristic_values: dict):
    # A min heap is used to implement the priority queue for the open list.
    # Entries in the heap are tuples of the form (cost, node), ensuring that the entry with the lowest cost is always smaller during comparisons.

    open_list  = [(heuristic_values[start], start)]
    closed_list = set()
    while open_list:
        cost, node = heappop(open_list)

        # The algorithm ends when the goal node has been explored, NOT when it is added to the open list.
        if node == goal:
            return cost

        if node in closed_list:
            continue

        closed_list.add(node)

        # Subtract the heuristic value as it was overcounted.
        cost -= heuristic_values[node]

        for neighbor, edge_cost in graph[node]:
            if neighbor in closed_list:
                continue

            # f(x) = g(x) + h(x), where g(x) is the path cost and h(x) is the heuristic.
            neighbor_cost = cost + edge_cost + heuristic_values[neighbor]
            heappush(open_list, (neighbor_cost, neighbor))

    return -1  # No path found

EXAMPLE_GRAPH = {
    'S': [('A', 4), ('B', 10), ('C', 11)],
    'A': [('B', 8), ('D', 5)],
    'B': [('D', 15)],
    'C': [('D', 8), ('E', 20), ('F', 2)],
    'D': [('F', 1), ('I', 20), ('H', 16)],
    'E': [('G', 19)],
    'F': [('G', 13)],
    'H': [('J', 2), ('I', 1)],
    'I': [('K', 13), ('G', 5), ('J', 5)],
    'J': [('K', 7)],
    'K': [('G', 16)]
}

# Node heuristic values (admissible heuristic values for the nodes)
EXAMPLE_HEURISTIC_VALUES = {
    'S': 7,
    'A': 8,
    'B': 6,
    'C': 5,
    'D': 5,
    'E': 3,
    'F': 3,
    'G': 0,
    'H': 7,
    'I': 4,
    'J': 5,
    'K': 3
}

EXAMPLE_RESULT = a_star_search(EXAMPLE_GRAPH, 'S', 'G', EXAMPLE_HEURISTIC_VALUES)
print(EXAMPLE_RESULT)
```

A* in Real-World AI Applications

1. GPS Navigation: These systems rely on A* to calculate the shortest and most efficient route between two locations while accounting for various factors like traffic, road conditions, and travel time.
2. Video Games: Like StarCraft and Age of Empires

3. Robotics
4. Network Routing: A* helps optimize the flow of data across interconnected networks by calculating the most efficient route for data packets.
5. Supply Chain Management
6. Urban Planning: map out **optimal infrastructure routes** for new roads, public transportation systems, and utilities.

Advantages

A* is guaranteed to find the optimal solution if the heuristic is admissible and consistent. It efficiently narrows down the search space by focusing on promising nodes.

Limitation

The algorithm's performance can degrade in large search spaces, as it requires significant memory to store all the nodes it explores.

Greedy Best-First Search

Greedy Best-First Search is an informed search algorithm that selects the next node to explore based solely on the heuristic value, **h(n)**, which estimates the cost to reach the goal. It always chooses the node that appears to be closest to the goal according to the heuristic, making it a fast algorithm but not always optimal.

Heuristic Function

$$f(x) = h(x)$$

Step-by-Step Guide

1. Initialize a tree with the root node being the start node in the open list.
2. If the open list is empty, return a failure, otherwise, add the current node to the closed list.
3. Remove the node with the lowest **h(x)** value from the open list for exploration.
4. If a child node is the target, return a success. Otherwise, if the node has not been in either the open or closed list, add it to the open list for exploration.

Implementation

```
def greedy_best_first_search_hierarchical(graph, start, goal, heuristic, region_map):
    priority_queue = []
    heapq.heappush(priority_queue, Node(start, heuristic[start]))

    visited = set()
    path = {start: None}

    while priority_queue:
        current_node = heapq.heappop(priority_queue).name

        if current_node == goal:
            return reconstruct_path(path, start, goal)

        visited.add(current_node)

        current_region = region_map[current_node]
        for neighbor in graph[current_node]:
            if neighbor not in visited and region_map[neighbor] == current_region:
                heapq.heappush(priority_queue, Node(neighbor, heuristic[neighbor]))
                if neighbor not in path:
                    path[neighbor] = current_node

        for neighbor in graph[current_node]:
            if neighbor not in visited and region_map[neighbor] != current_region:
                heapq.heappush(priority_queue, Node(neighbor, heuristic[neighbor]))
                if neighbor not in path:
                    path[neighbor] = current_node

    return None
def reconstruct_path(path, start, goal):
    current = goal
    result_path = []
    while current is not None:
        result_path.append(current)
        current = path[current]
    result_path.reverse()
    return result_path
```

Greedy Best-First Search in Real-World AI Applications

Web crawling and quick approximation.

Advantages

Greedy best-first search is fast and efficient in smaller search spaces, as it focuses on the most promising nodes first.

Limitation

The algorithm may not find the optimal solution because it only considers the heuristic and ignores the actual cost to reach a node; can get stuck in local minima.